

Q1: (6分) Please give 3 basic approaches for Multiprocessor (MP) operating systems.

請說明多處理器操作系統的 3 種基本實現方式。

👉請列出三種多處理器(MP)作業系統的基本方法。

答:速記:「各自做、主從制、全對稱」

三種基本方法為:

- (1) 每個 CPU 各自有自己的作業系統(Each CPU has its own OS)
- (2) 主從式多處理器(Master-Slave MP), 由一個 master CPU 控制系統, 其餘 slave CPU 執行任務。
- (3) 對稱式多處理器(SMP), 所有 CPU 平等, 共同執行同一個作業系統並共享資源。

Q2: (8分) What are the major pros and cons of non-blocking primitives in message passing systems?

在消息傳遞系統中, 非阻擋式原語的主要優缺點是什麼?

👉非阻塞(non-blocking)訊息傳遞機制的主要優點與缺點為何?

答:速記:「不卡快但不確定, 有額外成本」

非阻塞訊息傳遞的優點在於:發送者與接收者不需互相等待即可繼續執行, 提高系統並行度與效能, 並且較不容易發生 deadlock;

而其缺點包括:發送者無法立即確認訊息是否已被接收、在傳送完成前不能安全地修改訊息緩衝區, 此外還需要額外的通知機制與資料複製, 增加系統開銷。

Q3: (6分) For crossbar networks and omega networks (also called perfect shuffle), which network may occur blocking? Why do we still need the network that may occur blocking?

👉在 crossbar 與 omega(perfect shuffle)網路中, 哪一種可能發生 blocking? 為何仍然需要使用它?

答:速記:「Omega會卡但便宜, Crossbar不卡但很貴」

歐米伽網路在個通訊請求競爭同一交換器資源時, 會相互出現阻塞, 但因為其成本較低, 我們仍需要它。

交叉點網路(crossbar)需要 n^2 個交叉點, 而歐米伽網路只需要 $n/2 \log_2 n$ 個交換器即可, 因此在實務上更具可擴展性與經濟性。

Q4: (4分) A distributed system is described as scalable when it remains effective when there is a significant increase in the number of resources and the number of users. However, these systems sometimes face performance bottlenecks. How can these be avoided?

當分散式系統在資源數量和使用者的數量顯著增加的情況下仍能保持高效運作時, 就稱其具有可擴展性。然而, 這些系統有時會面臨效能瓶頸。如何避免這些問題?

👉在可擴展的分散式系統中, 如何避免效能瓶頸?

答:速記:「熱門資料→快取+複製分流」

在分散式系統中, 為了提升那些被大量使用的資源的效能, 可透過快取(Caching)和複製(Replication)機制來加以改善。

可透過 Caching(快取)與 Replication(複製)來避免效能瓶頸。快取可減少對熱門資源的重複存取, 降低延遲與負載;而複製則將資料分散到多個節點, 使多個請求能同時被處理, 避免單一節點成為瓶頸。

Q5: (8分) Please list and describe 4 consistency semantics for file sharing in a distributed system.

👉請列出並說明分散式系統中檔案共享的四種一致性語意(consistency semantics)。

答:速記:「全序、因果、單人順、關檔才更新」

常見的四種一致性語意為:

- (1) **Sequential consistency**(序列一致性):所有操作的結果等同於某個全域順序執行, 且每個 process 的操作順序保持不變;
- (2) **Causal consistency**(因果一致性):具有因果關係的寫入必須在所有節點上以相同順序被觀察, 但無關的並行寫入可有不同順序;
- (3) **FIFO consistency**(先進先出一致性):同一個 process 的寫入會依發生順序被所有節點看到, 但不同 process 間的寫入順序可不同;
- (4) **Session consistency**(會話一致性):在同一 client session 中的更新在檔案關閉後才對其他人可見, 並在重新開啟時取得最新版本。

Q6: (4分) Please list two disadvantages of the Direct Communication Model.

👉請列出直接通訊模型(Direct Communication Model)的兩個缺點。

答:速記:「名字綁死、事先要知道」

直接通訊模型的兩個主要缺點為：

- (1) 程序名稱耦合 (**tight coupling**)，當某個 process 名稱改變時，所有與其通訊的程式都必須修改；
- (2) 需事先得知對方身份 (**lack of flexibility**)，傳送訊息前必須明確知道接收者的名稱，降低系統的彈性與擴展性。

Q7: (8分)

In multicasting systems, there are 4 levels of reliability degrees that can be specified by a sender. Please list and describe them briefly.

✎ 在多播 (multicasting) 系統中，sender 可指定四種可靠度等級，請列出並簡述。

答：速記：「0、1、M、All (從不回到全回)」

四種可靠度等級為：

- (1) **0-reliable**(0 級可靠性)：不期望任何接收者回應，純單向傳送 (例如時間廣播)；
- (2) **1-reliable**(1 級可靠性)：只需任一接收者回應即可，常用於選擇最快回應者；
- (3) **M-out-of-N reliable**(M 級可靠性)：共有 N 個接收者，只需其中 M 個回應即可 ($1 < M < N$)，常用於容錯與冗餘系統 (如 RAID)；
- (4) **All-reliable**(完全可靠)：要求所有接收者都回應，確保訊息完全送達所有節點。

Q8: (4分) What is the cache coherence problem?

✎ 什麼是快取一致性問題 (cache coherence problem)？

答：速記：「多份資料，一處改要全部同步」

在多處理器系統中，當同一筆資料被複製到多個 CPU 的 cache 中時，如果某一個 CPU 修改了該資料，其他 cache 中的副本可能仍是舊值，導致資料不一致；因此系統必須透過 **cache coherence protocol** 來確保所有 cache 對該資料的內容保持一致，例如在寫入時通知其他 cache 失效或更新其內容。

Q9: (8分) Suppose that the switch 2B and 3C in the omega network of Fig. 8-5 fail. What CPUs are cut off from which memories? Please list all pairs of (CPU, MEM) that cannot communicate.

✎ 若 Omega network 中的 switch 2B 與 3C 故障，哪些 CPU 與哪些 Memory 無法連通？請列出所有 (CPU, MEM) 配對。

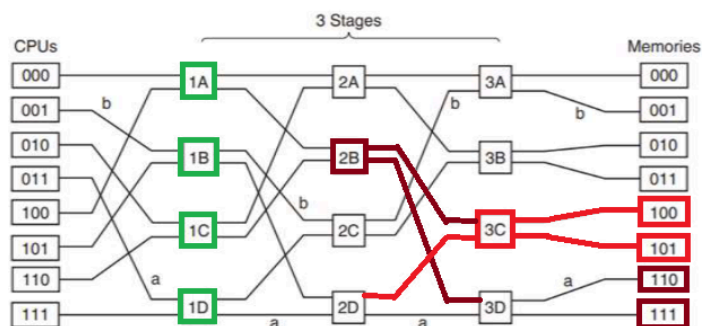


Figure 8-5. An omega switching network.

答：速記：「switch 壞 → 所有經過它的路徑全斷」

當 2B 與 3C 同時故障時，無法通訊的 (CPU, MEM) 共計 24 組 (16 + 4)，組合如下。

1A(0) = (000, 100), (000, 101), (000, 110), (000, 111)

1A(1) = (100, 100), (100, 101), (100, 110), (100, 111)

1C(0) = (010, 100), (010, 101), (010, 110), (010, 111)

1C(1) = (110, 100), (110, 101), (110, 110), (110, 111)

1B(0) = (001, 100), (001, 101)

1B(1) = (101, 100), (101, 101)

1D(0) = (011, 100), (011, 101)

1D(1) = (111, 100), (111, 101)

Q10: (6分) Use the omega network, please give the routes for the following CPU memory pairs.

答：

(a) from CPU 010 to Memory 101 : (A) 010 → 1C → 2B → 3C → 101

(b) from CPU 100 to Memory 111 : (B) 100 → 1A → 2B → 3D → 111

(c) from CPU 111 to Memory 010 : (C) 111 $\rightarrow$ 1D $\rightarrow$ 2C $\rightarrow$ 3B $\rightarrow$ 010

Q11: (6分) For each of the topologies of Fig. 8-16, what is the bisection width of the interconnection network?

👉對於圖 8-16 中的各種拓撲，其網路的**雙分寬度**(bisection width)為何？

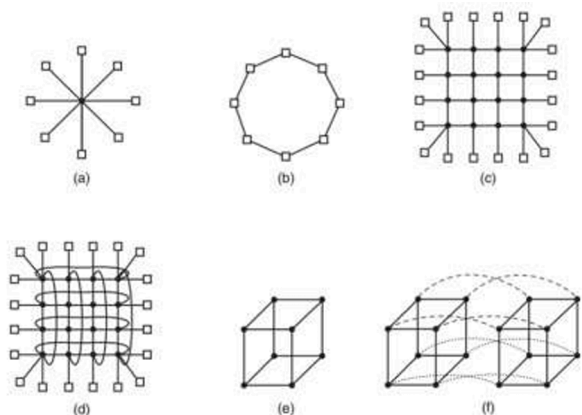


Figure 8-16. Various interconnect topologies. (a) A single switch. (b) A ring. (c) A grid. (d) A double torus. (e) A cube. (f) A 4D hypercube.

答: (a) $O(N)$ or 4 (b) 2 (c) $O(\sqrt{N})$ or 4 (d) 8 (e) $O(N)$ or 4 (f) 8

Q12: (4分) What's the difference between Open and Closed Groups?

👉(4%) 公開群組和封閉群組有什麼區別？

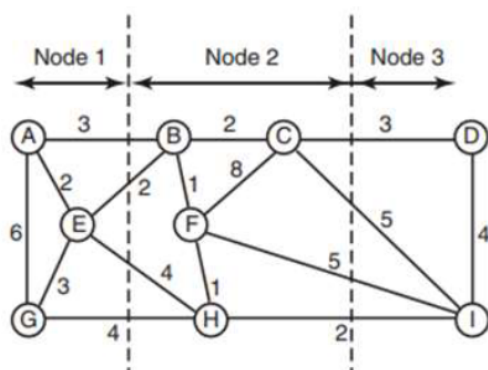
答: 速記: 「Closed 只限內部, Open 人人可發」

Closed Group(封閉群組)是指只有群組內的成員才能對整個群組發送訊息，外部程序無法對群組進行廣播。

Open Group(開放群組)則允許系統中任何程序(不論是否為成員)都可以向整個群組發送訊息。

Q13: (5分) Consider the processor allocation of the following figure. What is the total weight of the external traffic? If process F is moved from node 2 to node 3, what is the total weight of the external traffic now?

👉請考慮下圖中的處理器分配情況。外部流量的總權重是多少？如果將處理器 F 從節點 2 移動到節點 3, 那麼此時外部流量的總權重又是多少呢？



答:

- 外部流量總權重是多少？

Node 1: A, E, G

Node 2: B, C, F, H

Node 3: D, I

跨節點邊:

Node 1 $\leftrightarrow$ Node 2: AB = 3, BE = 2, EH = 4, GH = 4, 總和 = 13

Node 2 $\leftrightarrow$ Node 3: CD = 3, CI = 5, FI = 5, HI = 2, 總和 = 15

所以: Total external traffic = 13 + 15 = 28

- 將 F 從 Node 2 移到 Node 3 後？

Node 1: A, E, G

Node 2: B, C, H

Node 3: F, D, I

跨節點邊:

Node 1 $\leftrightarrow$ Node 2: AB = 3, BE = 2, EH = 4, GH = 4, 總和 = 13

Node 2 ↔ Node 3: BF = 1, CF = 8, FH = 1, CD = 3, CI = 5, HI = 2, 總和 = 20

所以: New total external traffic = 13 + 20 = 33

Q14: (6分) Describe how to implement a binary semaphore by using the **Linda** system (You can use the following command: out, in).

👉請說明如何使用 Linda 系統來實作二進位信號量。可以使用以下指令: out、in。

答:

- 初始化: out("sem")
- wait() / P 操作: in("sem")
- signal() / V 操作: out("sem")

Q15: (8分) Please list 4 common failures that may happen in RPC model.

👉(8%) 請列出在 RPC 模型中可能出現的 4 種常見故障。

答: 速記:「找不到、送不到、回不來、伺服器掛掉」

- (1) 無法找到伺服器 / 伺服器當機 / 版本不匹配 / 其他各種錯誤。
- (2) 請求訊息遺失。
- (3) 回覆訊息遺失。
- (4) 伺服器在收到請求後, 在處理後或處理中崩潰。
- (5) 用戶端在發送請求後當機。

Q16: (6分) Please list 3 possible problems for passing parameters in RPC calls.

👉(6%) 請列出在 RPC 調用中傳遞參數時可能出現的 3 種問題。

答: 速記:「指標、陣列、參數數量不固定」

- (1) 傳遞指針: 指標指向的是本地記憶體位址, 無法直接在遠端使用。
- (2) 傳遞無限制大小的陣列: 在 C 語言中, 陣列的大小在編譯時可能是不確定的, RPC 傳遞需明確大小, 否則難以序列化。
- (3) 參數數量無限制: C 語言中的 printf 函數, 其所需的參數數量並非固定不變, RPC 難以處理可變參數。
- (4) 將全局變數作為參數傳遞。

Q17: (6分) Please list 3 common approaches for identifying a process in direct communication based message passing systems.

👉(6%) 請列出在基於消息傳遞的直接通訊系統中, 用於識別某個進程的 3 種常用方法。

答:

- (1) **Hostname + Local-ID**: 使用主機名稱加上該主機內的 process ID 作為唯一識別。
 - (2) **Hostname (original) + Local-ID + Hostname (current)**: 同時記錄原始主機與目前所在主機, 支援 process migration, 並透過轉送機制找到新位置;
 - (3) **Name server-based approach**: 使用全域名稱服務將人類可讀名稱對應到目前的 process ID 或位置, 提供動態查詢與彈性。
-

Q1: Using Test-and-Set-Lock (TSL) instruction to implement mutex locks for MPs needs to lock the system bus. However, this may increase the bus contention. Please propose two methods to reduce the **bus contention**.

👉(4%) 使用“測試並設置鎖”(TSL)指令來實現多處理器環境中的互斥鎖時，需要鎖定系統總線。不過，這可能會增加總線競爭的情況。請提出兩種減少總線競爭的方法。

答:速記:「先讀再鎖, 失敗退避」

可採用兩種方法降低 bus contention:

- (1) 在 TSL 指令之前先以普通 **read** 檢查 **lock**, 再必要時才使用 **TSL (test-and-test-and-set)**, 讓 CPU 在本地 cache 中忙等, 避免頻繁鎖住 bus;
- (2) 使用 **backoff** 機制(如 **exponential backoff**), 當取得 lock 失敗時延遲一段時間再重試, 減少多個 CPU 同時競爭 bus 的情況。

Q2: What are Sender-initiated Load Balancing and Receiver-initiated Load Balancing algorithms? Please give brief descriptions and compare their differences.

👉(6%) 何謂發送端主導的負載平衡算法和接收端主導的負載平衡算法? 請簡要說明並比較兩者的差異。

答:速記:「忙的送(push), 閒的拿(pull); 輕載送, 重載拿」

在分散式系統中,

- Sender-initiated(發送端主導負載平衡)是指當某個節點負載過重時, 主動將工作(tasks)推送(push)到其他節點, 適合在系統整體負載較低時使用, 因為較容易找到空閒節點;
- 而 Receiver-initiated(接收端主導負載平衡)則是當某個節點空閒時, 主動向其他節點拉取(pull)工作, 適合在系統整體負載較高時使用, 因為較容易找到忙碌節點來分擔負載。
- 兩者的主要差異在於觸發條件(過載 vs 空閒)、行為方式(push vs pull)以及適用的系統負載情境(light load vs heavy load)。

Q3: Affinity scheduling reduces cache misses. Does it also reduce TLB misses? Does it reduce page faults?

👉親和性排程(Affinity scheduling)可以降低 cache miss, 那它是否也能降低 TLB miss? 是否能降低 page fault?

答:速記:「同 CPU → cache/TLB 變好; page fault 不變」

Affinity scheduling(親和性排程)讓 process 盡量在同一 CPU 上執行, 因此可以降低 **TLB misses**, 因為該 CPU 的 TLB 仍保留該 process 的地址轉換資訊;但它無法降低 **page faults**, 因為 page fault 是由資料是否存在於主記憶體決定, 與是否在同一 CPU 執行無關。

Q4: What are the major pros and cons of master-slave multiprocessors? Please list 2 of them, respectively.

👉主從式多處理器(Master-Slave MP)的主要優點與缺點為何? 請各列兩點。

答:速記:「好管但會卡, 主壞全掛」

優點:

- (1) 設計簡單, 由 master CPU 負責排程與控制, 其餘 slave CPU 只執行任務, 系統結構清楚;
- (2) 容易管理與同步, 所有關鍵操作集中在 master, 避免複雜的同步問題。

缺點:

- (1) 效能瓶頸, 所有請求都需經過 master, 容易成為效能瓶頸;
- (2) 單點故障(**single point of failure**), 一旦 master 當機, 整個系統可能無法運作。

Q5: What are the number of switches for crossbar network and Omega network? Consider n CPUs and n memory modules.

👉對於 n 個 CPU 與 n 個記憶體模組, Crossbar 與 Omega network 分別需要多少個 switches?

答:速記:「Crossbar = n^2 , 全連接; Omega = $n/2 \log_2 n$, 分層省成本」

Crossbar: n^2

Omega: $\frac{n}{2} \log_2 n$

在 **Crossbar network** 中, 需要 n^2 個 crosspoints(switches), 因為每個 CPU 都需能直接連到每個 memory; 而在 **Omega network** 中, 需要 $n/2 \log_2 n$ 個 switches, 因為共有 $\log_2 n$ 層, 每層有 $n/2$ 個 switch。

Q6: Please list and describe 3 types of transparency in a distributed system.

👉(6%) 請列出並說明分散式系統中的 3 種透明度類型。

答:「存取、位置、並行」

1. **存取透明度**: 讓使用者能以相同的操作方式來存取本地或遠端的資訊資源。
2. **位置透明度**: 讓使用者無需知道資源的實際位置即可存取它們。
3. **並行透明度**: 允許多個程序同時使用相同的共享資源, 且彼此不會互相干擾。
4. **複製透明度**: 允許使用多個資源副本來提升系統的可靠性與效能, 而使用者或應用程式無需知道這些副本的存在。
5. **遷移透明度**: 讓資源能在分散式系統內部被移動, 而不影響使用者或應用程式的運作。
6. **故障透明度**: 能夠隱藏系統中的故障。即使某些組件出現故障, 使用者及應用程式仍可繼續執行其任務。
7. **效能透明度**: 讓分散式系統及應用程式能夠根據需求進行重新配置, 而不必改變系統結構或應用程式算法。

Q7: Please give 2 differences between the normal MPs and Chip-level MPs.

👉(4%) 請說明一般多處理器(Normal MPs)與晶片級多處理器(Chip-level MPs, CMP)的兩個差異。

答: 速記:「分開慢又貴, 同晶片快又省」

兩者主要差異為:

- (1) 硬體整合方式: Normal MPs 是多個獨立 CPU(多晶片)透過匯流排或網路連接, 而 Chip-level MPs(CMP)則是多個核心整合在同一顆晶片上;
- (2) 通訊延遲與效率: Normal MPs 需透過外部 bus 通訊, 延遲較高且成本較大, 而 CMP 在晶片內通訊(on-chip), 延遲較低、效率較高。

Q8: What makes blocking call a better way to message sending than non-blocking call in multithreaded system? List at least 2 reasons.

(4%) 在多執行緒系統中, 為何阻擋式(blocking call) 通話比非阻擋式(non-blocking call) 通話更適合用於傳送訊息? 請列出至少兩個理由。

答:

- (1) 簡化同步機制, 因為呼叫會等待傳送或接收完成, 不需要額外的 polling、callback 或通知機制, 程式設計較簡單且不易出錯;
- (2) 避免資料不一致或 **race condition**, blocking 能確保訊息已完成傳送後才繼續執行, 避免 buffer 尚未送出就被修改;
- (3) 減少額外控制開銷, 不需額外的中斷或通知機制來確認訊息狀態。

Q9: Suppose that the wire between switch 2C and switch 3A in the omega network breaks. Who are cut off from whom?

在 Omega network 中, 若 switch 2C 與 3A 之間的連線斷掉, 哪些 CPU 與哪些 Memory 之間會無法連通?

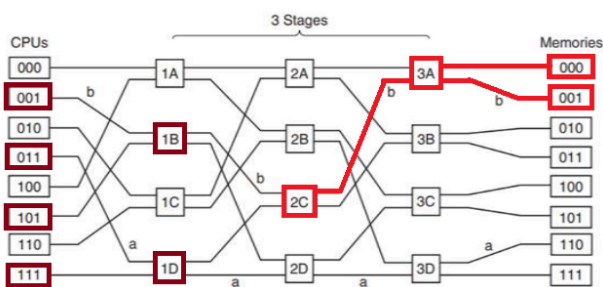


Figure 8-5. An omega switching network.

Ans: CPU 001、011、101、111 與記憶體 000 和 001 是斷開連接的。

Q10: (6%) Use the omega network in the above, please give the routes for the following CPU memory pairs. (hint: CPU 000 -> 1A -> 2A -> 3A -> Memory 000)

答:

- | | |
|-------------------------|--------------------------|
| (a) 從 CPU 010 到記憶體 111: | (a) 010->1C->2B->3D->111 |
| (b) 從 CPU 100 到記憶體 100: | (b) 100->1A->2B->3C->100 |
| (c) 從 CPU 111 到記憶體 001: | (c) 111->1D->2C->3A->001 |

Q11: For each of the topologies of Fig. 8-16, what is the **diameter** of the interconnection network? Count all hops (host-router and router-router) equally for this problem.

(6%) 對於圖 8-16 中的每一種拓撲結構，**互聯網絡的直徑是多少**？在計算時，請將所有的跳數（包括主機到路由器的跳數以及路由器到路由器的跳數）一視同仁地計算在內。

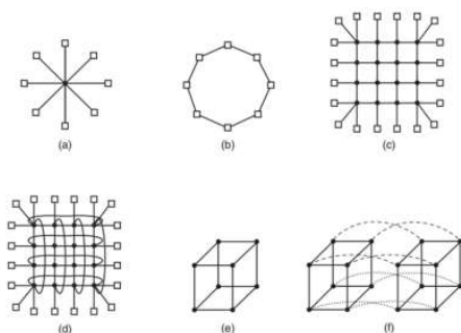


Figure 8-16. Various interconnect topologies. (a) A single switch. (b) A ring. (c) A grid. (d) A double torus. (e) A cube. (f) A 4D hypercube.

Ans: (a) 2 (b) 4 (c) 8 (d) 5 (e) 3 (f) 4

Diameter (直徑): 任意兩個節點之間的最長最短路徑 (**max shortest path**), 單位是: hop 數

Q12: Please list and describe the 3 types of redundancy.

👉請列出並說明三種冗餘 (redundancy) 類型。

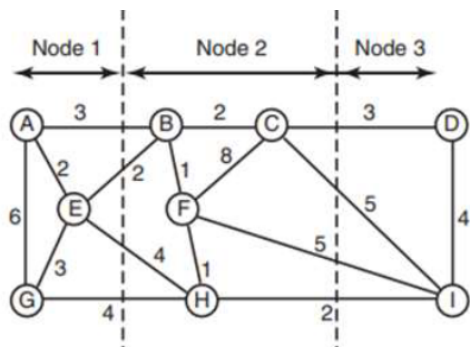
答:

三種基本冗餘為:

- (1) **Information redundancy** (資訊冗餘): 加入額外資料 (如 parity、ECC) 來偵測或修正錯誤;
- (2) **Time redundancy** (時間冗餘): 將同一操作重複執行多次, 以確認結果正確或抵抗暫時性錯誤;
- (3) **Physical redundancy** (實體冗餘): 使用多個硬體元件 (如多台伺服器或多個 CPU) 來執行相同任務, 以提升系統可靠度與容錯能力。

Q13: Suppose processes C and F are moved from node 2 to node 3. What is the total weight of the external traffic now?

👉(3%) 請考慮下圖中的處理器分配情況。假設過程 C 和 F 從節點 2 移動到了節點 3。那麼, 此時外部流量的總重量是多少呢?



Node 1: A, E, G

Node 2: B, H

Node 3: C, F, D, I

- Node 1 ↔ Node 2
AB=3, BE=2, EH=4, GH=4 合計= 13
- Node 2 ↔ Node 3
BC=2, BF=1, HF=1, HI=2, 合計= 6

所以新的總權重為 19: **13 + 6 = 19**

Q14: Please describe UMA and NUMA in the context of multi-processor systems.

👉(4%) 請在多處理器系統的背景, 說明 UMA 和 NUMA 的原理。(多處理器系統)

答:

在多處理器系統中, UMA (Uniform Memory Access) 指所有 CPU 存取記憶體延遲是一致的, 記憶體通常為集中式共享, 因此設計簡單但擴展性較差;

而 ****NUMA (Non-Uniform Memory Access)**** 則是每個 CPU 擁有本地記憶體, 存取本地記憶體較快、遠端記憶體較慢, 雖然設計較複雜, 但具有較好的可擴展性與效能。

Q15: Describe how to implement a counting semaphore using the Linda system (using commands: out, in).

(6%) 請說明如何使用 Linda 系統來實作計數信號量。(您可以使用以下指令: out、in。)答: (每題 2%)

答:

初始化: 執行 out("sem") N 次。

wait(): 進入 "sem" 狀態;

signal(): 退出 "sem" 狀態。

Q16: Please give 4 possible methods for notifying a non-blocking receiver that the desired message has arrived.

👉(8%) 請列出 4 種可行的方法, 用來通知非阻擋式接收器: 所需的消息已經到達。

(1) **Polling** (輪詢): receiver 定期主動檢查訊息是否到達;

(2) **Interrupt** (中斷): 訊息到達時由系統中斷通知 receiver;

(3) **Callback / Upcall** (回呼): receiver 先註冊處理函式, 訊息到達後由系統呼叫;

(4) **Signal / Event notification** (訊號或事件通知): 利用 signal、event flag 或 condition variable 通知 receiver。

Q17: Please give 3 basic failures that may happen in message exchanges of client-server model.

👉(6%) 請說明在客戶端-伺服器模式的資訊交換中, 可能出現的 3 種基本故障。

答: 速記:「送不到、回不來、伺服器掛掉」

(1). LostReq: 請求訊息遺失。

(2). LostResp: 回應訊息遺失。

(3). SerCrash: 請求執行失敗。

Q18: Please give brief descriptions about spin and block with context switch and provide in what scenarios it is appropriate to use spin and block with context switch in an SMP system, respectively.

👉(4%) 請簡要說明在 SMP 系統中, 伴隨著上下文切換的「旋轉等待」和「阻塞等待」機制分別適用於何種情況。每題 2 分。

答: 速記:「短等 spin, 長等 block」

- 在 SMP 系統中, spin(自旋)是指執行緒持續在 CPU 上忙等(busy waiting)檢查鎖是否可用, 不進行 context switch, 因此延遲低但會消耗 CPU 資源, 適合用於臨界區很短或預期等待時間很短的情況;
 - 而 block(阻塞)則是當無法取得資源時讓執行緒進入睡眠, 並透過 context switch 切換到其他執行緒執行, 雖然有切換成本但能節省 CPU, 適合用於等待時間較長或競爭較激烈的情況。
-

Q19: Please describe the two common data transfer models in a Distributed File System.

👉(4%) 請說明分散式檔案系統中兩種常見的資料傳輸模式。

答: 速記:「整包下載本地做 vs 每次都遠端做」

在分散式檔案系統中,

第一種是 **Upload/Download Model**(上傳/下載模型), 當使用者開啟檔案時會先將整個檔案下載到本地端, 之後所有操作在本地進行, 關閉時再將更新後的檔案上傳回伺服器, 優點是可減少網路通訊並提升效能, 但可能有一致性延遲;

第二種是 **Remote Access Model**(遠端存取模型), 檔案始終保留在伺服器上, 每次 read/write 都透過網路直接操作遠端資料, 優點是即時一致性較好, 但網路負擔較大且延遲較高。

Q20: In publish-subscribe systems, explain how **channel-based** approaches can be implemented using a group communication service.

👉(3%) 在發布-訂閱式系統中, 請說明如何利用群組通訊服務來實現以通道為基礎的處理方式?

答: 速記:「channel(頻道) = group(群組); publish(發布) = multicast(多播); subscribe(訂閱) = join(加入)」

在 channel-based 的 publish-subscribe 系統中, 每一個 **channel** 可以對應到一個群組(**group**)。發布者(publisher)將訊息送到該 channel, 實際上就是對該群組進行 **multicast**(群播); 而訂閱者(subscriber)則透過加入(join)或離開(leave)對應的群組來表達訂閱或取消訂閱。如此一來, 群組通訊服務負責將訊息自動分發給所有群組成員, 從而實現 channel-based 的發布-訂閱機制。

Q21: Please list 3 major advantages of publish/subscribe middleware model over the point-to-point model.

(6%) 請列出發布/訂閱中介模型相較於點對點模型的三個主要優點。

答：

三個主要優點為：

- (1) **Decoupling**(解耦合) : publisher 與 subscriber 不需知道彼此的存在或位置(時間、空間、同步解耦), 提升系統彈性;
- (2) **Scalability**(可擴展性) : 支援一對多與多對多通訊, 新增或移除節點不需修改既有程式;
- (3) **Flexibility**(彈性/動態訂閱) : subscriber 可動態加入或離開, 並依需求訂閱特定事件或主題。

2024-09: Omega Network 斷線影響

Suppose that the wire between switch 2C and switch 3A in the omega network breaks. Who are cut off from whom?

在 Omega network 中, 若 switch 2C 與 3A 之間的連線斷掉, 哪些 CPU 與哪些 Memory 之間會無法連通?

Q (中): 2C → 3A 的「連線(wire)」斷掉。**Q (EN):** wire between switch 2C and 3A breaks

2025-09: (8%) Suppose that the switch 2B and 3C in the omega network of Fig. 8-5 fail. What CPUs are cut off from which memories? Please list all pairs of (CPU, MEM) that cannot communicate.

👉 假設圖 8-5 中 omega 網路中的交換器 2B 和 3C 故障。哪些 CPU 會被切斷到哪些記憶體? 請列出所有無法通訊的(CPU、MEM)組合。

Q (中): 整個 **switch 2B 與 3C** 壞掉。**Q (EN):** switch 2B and 3C fail

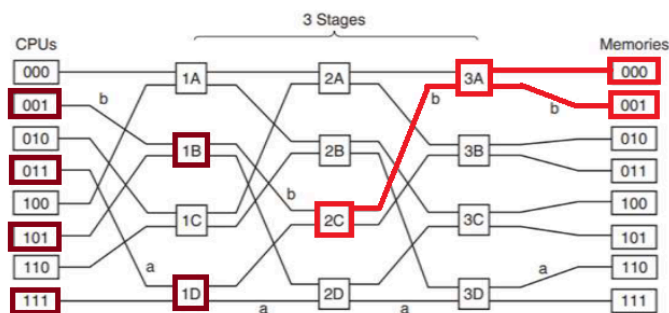


Figure 8-5. An omega switching network.

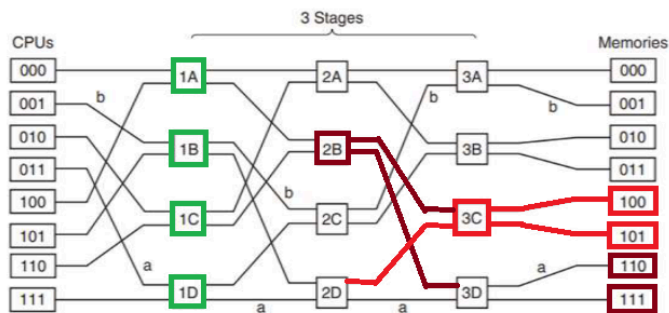


Figure 8-5. An omega switching network.

2024-11:

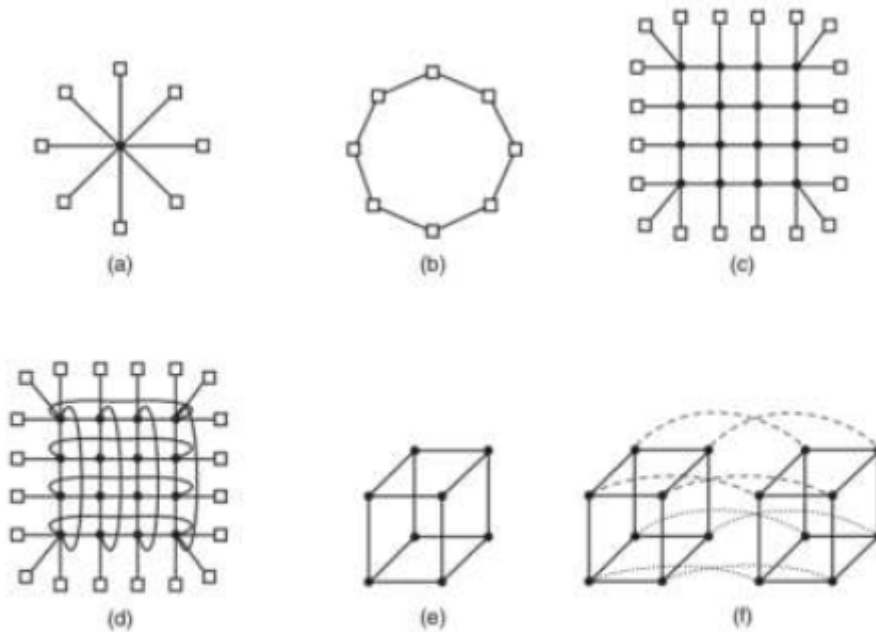


Figure 8-16. Various interconnect topologies. (a) A single switch. (b) A ring. (c) A grid. (d) A double torus. (e) A cube. (f) A 4D hypercube.

(6%) For each of the topologies of Fig. 8-16, what is the **diameter** of the interconnection network? Count all hops (host-router and router-router) equally for this problem.

(6%) 對於圖 8-16 中的每一種拓撲結構，**互聯網路的直徑是多少**？在計算時，請將所有的跳數（包括主機到路由器的跳數以及路由器到路由器的跳數）一視同仁地計算在內。

Ans: (a) 2 (b) 4 (c) 8 (d) 5 (e) 3 (f) 4

Diameter(直徑): 任意兩個節點之間的最長最短路徑(**max shortest path**), 單位是: hop 數
重點:

- 衡量「最壞情況延遲(latency)」
- 路徑越短 → 傳輸越快

2025-11: (6%) For each of the topologies of Fig. 8-16, what is the **bisection** width of the interconnection network?

👉 對於圖 8-16 的每種拓撲，**互聯網路的二截寬是多少**？

答: (a) $O(N)$ or 4 (b) 2 (c) $O(\sqrt{N})$ or 4 (d) 8 (e) $O(N)$ or 4 (f) 8

(a) Single switch(星狀拓撲): Star network 的 bisection width = $N/2 = 4$

(b) Ring(環狀拓撲): Ring 無論有多少節點, bisection width = 2。

(c) Grid(網格拓撲): 在 $k * k = n$ 的網格中 $\sqrt{n} = 4$

(d) Double torus(雙環面拓撲): $= 2k = 8$

(e) 立方體 (3D): 4

(f) 超立方體 (4D): 8

Bisection Width(雙分寬度): 將網路切成兩半所需切斷的最少連線數, 衡量「最小切割容量」